System Overview & Architecture

Fire Simulation System

27 stycznia 2026

1 Wprowadzenie

1.1 Cel dokumentu

Dokument zawiera opis architektury systemu symulacji pożarów lasu, jego składników, przepływu danych oraz sposobu uruchomienia całego systemu.

1.2 Zakres dokumentu

2 Architektura systemu

2.1 Przegląd architektury

System symulacji pożarów lasu zbudowany jest w architekturze mikroserwisów, składającej się z czterech głównych modułów oraz infrastruktury wspierającej:

- **FFBackend** (fire-backend) – serwis Java/Spring Boot odpowiedzialny za logikę biznesową i koordynację
- **FFSim** (fire-simulation) – silnik symulacji napisany w Pythonie
- **FFSup** (fire-support) – serwis wsparcia decyzyjnego wykorzystujący MCTS
- **FFVis** (fire-visualization) – aplikacja frontendowa do wizualizacji
- **FFConf** (fire-configurations) – serwis zarządzania konfiguracjami lasu

2.2 Technologie i narzędzia

- **Backend:** Java 17, Spring Boot, Maven
- **Symulacja:** Python 3.x, asyncio
- **Frontend:** TypeScript, React, MapLibre GL
- **Komunikacja:** RabbitMQ (topic exchange), HTTP/REST, Server-Sent Events (SSE)
- **Baza danych:** MongoDB
- **Orchestracja:** Docker, Docker Compose
- **Logowanie:** Grafana, Loki, Promtail (w trakcie implementacji)

3 Komponenty systemu

3.1 FFBackend (fire-backend)

3.1.1 Odpowiedzialności

- Zarządzanie stanem symulacji (sektory, brygady, patrole)
- Agregacja danych z symulacji
- Udostępnianie REST API dla frontendu
- Strumieniowanie stanu symulacji przez SSE
- Przekazywanie komend z frontendu do symulacji
- Publikowanie zagregowanych danych do serwisu wsparcia

3.1.2 Zależności

- Wymaga: RabbitMQ, MongoDB (opcjonalnie)
- Komunikuje się z: FFSim (HTTP + RabbitMQ), FFSup (RabbitMQ), FFVis (HTTP/SSE)
- Port: 8181 (domyślnie)

3.2 FFSim (fire-simulation)

3.2.1 Odpowiedzialności

- Wykonanie symulacji fizycznej pożaru lasu
- Modelowanie rozprzestrzeniania się ognia
- Zarządzanie agentami (brygady pożarnicze, patrole leśne)
- Aktualizacja stanu sektorów (poziom ognia, spalenia, gaszenia)
- Publikowanie danych telemetrycznych (czujniki, agenci, sektory)
- Odbieranie komend sterujących od backendu

3.2.2 Zależności

- Wymaga: RabbitMQ
- Komunikuje się z: FFBackend (HTTP + RabbitMQ)
- Port: 5000 (domyślnie)

3.2.3 Ważna uwaga

Symulator jest osobnym bytem w systemie. Wszystkie decyzje podejmowane są przez system (FFBackend, FFSup), a symulator jedynie wykonuje komendy i symuluje fizykę pożaru.

3.3 FFSup (fire-support)

3.3.1 Odpowiedzialności

- Generowanie rekomendacji decyzyjnych przy użyciu MCTS
- Analiza stanu symulacji
- Publikowanie rekomendacji do backendu/frontendu
- Zarządzanie pulą workerów MCTS
- Konwersja stanu symulacji do formatu wymaganego przez algorytmy

3.3.2 Zależności

- Wymaga: RabbitMQ
- Komunikuje się z: FFBackend (RabbitMQ)
- Port: brak (tylko RabbitMQ)

3.4 FFVis (fire-visualization)

3.4.1 Odpowiedzialności

- Wizualizacja mapy lasu i sektorów
- Wyświetlanie pozycji brygad i patroli
- Prezentacja rekomendacji decyzyjnych
- Interakcja użytkownika (wysyłanie komend)
- Wyświetlanie danych z czujników

3.4.2 Zależności

- Wymaga: FFBackend (HTTP/SSE)
- Port: 8080 (domyślnie)

3.5 FFConf (fire-configurations)

3.5.1 Odpowiedzialności

- Zarządzanie konfiguracjami lasu w MongoDB
- Udostępnianie API do CRUD konfiguracji
- Walidacja konfiguracji JSON
- Przechowywanie hierarchicznej struktury konfiguracji

3.5.2 Zależności

- Wymaga: MongoDB
- Port: 31415 (domyślnie)

4 Komunikacja między modułami

4.1 Protokoły komunikacji

4.1.1 HTTP/REST

Używany do:

- Komunikacji FFVis ↔ FFBackend
- Komunikacji FFBackend ↔ FFSim (komendy, konfiguracja)
- Komunikacji FFVis ↔ FFConf (pobieranie konfiguracji)

4.1.2 RabbitMQ (Topic Exchange)

Używany do:

- Publikacji danych telemetrycznych z FFSim do FFBackend
- Publikacji zagregowanych danych z FFBackend do FFSup
- Publikacji rekomendacji z FFSup do FFBackend
- Kolejkowania komend gdy FFSim jest niedostępny

4.1.3 Server-Sent Events (SSE)

Używany do:

- Strumieniowania stanu symulacji z FFBackend do FFVis
- Jednokierunkowa komunikacja w czasie rzeczywistym

4.2 Exchange RabbitMQ

Wszystkie kolejki używają exchange typu `topic` o nazwie `fire-simulation-exchange` (konfigurowalne przez zmienną środowiskową `RABBITMQ_EXCHANGE`).

5 Uruchomienie systemu

5.1 Wymagania

- Docker i Docker Compose
- Minimum 4GB RAM
- Porty: 8080, 8181, 5000, 31415, 27017, 5672, 15672

5.2 Uruchomienie przez Docker Compose

5.2.1 Uruchomienie wszystkich serwisów

```
docker compose -f docker-compose.yml up --build
```

5.2.2 Uruchomienie wybranych serwisów

```
# Najpierw infrastruktura
docker compose up --build rabbitmq-service mongo

# Nast pnie serwisy aplikacyjne
docker compose up --build fire-backend-service
docker compose up --build fire-simulation-service
docker compose up --build fire-configuration-service
docker compose up --build fire-visualization-service
```

5.3 Kolejność uruchamiania

Zalecana kolejność uruchamiania serwisów:

1. **MongoDB** – baza danych dla konfiguracji
2. **RabbitMQ** – broker komunikatów
3. **FFConf** – serwis konfiguracji (wymaga MongoDB)
4. **FFBackend** – backend (wymaga RabbitMQ, opcjonalnie MongoDB)
5. **FFSim** – symulator (wymaga RabbitMQ)
6. **FFSup** – serwis wsparcia (wymaga RabbitMQ)
7. **FFVis** – frontend (wymaga FFBackend)

5.4 Sprawdzanie statusu

- **FFBackend:** `http://localhost:8181/actuator/health`
- **FFConf:** `http://localhost:31415/health`
- **RabbitMQ Management:** `http://localhost:15672`
- **FFVis:** `http://localhost:8080`

6 Architektura agentowa

6.1 Model agentów

System wykorzystuje architekturę agentową, gdzie agenci reprezentują autonomiczne jednostki działające w symulacji.

6.1.1 Definicja agenta

Agent w systemie to:

- **Instancja runtime:** obiekt w pamięci reprezentujący jednostkę (brygada pożarnicza, patrol leśny)
- **Cykl życia:** inicjalizacja → decyzja → akcja → raport
- **Miejsce w architekturze:** agenci "żyją"w FFSim, ale są zarządzani przez FFBackend

6.1.2 Typy agentów

- **FireBrigade** – brygada pożarnicza
  - Stany: AVAILABLE, TRAVELLING, EXTINGUISHING
  - Zadania: gaszenie pożarów w sektorach
  - Decyzje: wybór sektora do gaszenia (może być LLM-driven)
- **ForesterPatrol** – patrol leśny
  - Stany: AVAILABLE, TRAVELLING, PATROLLING
  - Zadania: patrolowanie sektorów, wykrywanie zagrożeń
  - Decyzje: wybór sektora do patrolowania (heurystyka, może być LLM-driven)

6.2 Separacja logiki decyzyjnej

Ważne: Agent jest oddzielony od logiki decyzyjnej. Logika decyzyjna może być:

- Deterministyczna (reguły, heurystyki)
- MCTS (Monte Carlo Tree Search)
- LLM-driven (wymienna strategia decyzyjna)

LLM jest traktowany jako **jedna z implementacji** modułu decyzyjnego.

7 Logi systemu

7.1 Struktura logów

Wszystkie serwisy logują do plików w katalogu `logs/`:

- `logs/fire-backend/application.log`
- `logs/fire-simulation/application.log`
- `logs/fire-configuration/application.log`

8 Wersjonowanie i kompatybilność

8.1 Wersjonowanie komunikatów

Obecnie system nie implementuje formalnego wersjonowania komunikatów. Zalecane jest dodanie wersji do:

- Kontraktów RabbitMQ (pole `version` w nagłówkach)
- REST API (prefiks `/api/v1/`)
- Formatów JSON konfiguracji

9 Ograniczenia systemu

9.1 System alertów

System **nie posiada** mechanizmu alertów w czasie rzeczywistym. W przyszłości można dodać koncepcyjny mechanizm alertów jako osobny moduł.

9.2 Charakter symulacyjny

Wszystkie dane w systemie są generowane przez symulator. System nie korzysta z:

- Realnych czujników środowiskowych
- Realnych danych meteorologicznych (oprócz symulowanego wiatru)
- Realnych jednostek pożarniczych

10 Podsumowanie

System symulacji pożarów lasu jest zbudowany w architekturze mikroserwisów z wyraźnym podziałem odpowiedzialności.